

Python Cheatsheet

MATH60033A · Session 04 · Logistic regression

Thierry Warin, PhD · HEC Montréal · A2026

Everything you need to type, in one place. The first three sections are the same every week; the last one is the Python from session 3.

Starting Python in VS Codium

Open the course folder once — **File > Open Folder**, then pick the repository. Everything below assumes you are inside it.

Open a terminal with **View > Terminal** (Ctrl + ` on Windows and Linux, Cmd + ` on macOS). Create the environment the first time only:

```
python3 -m venv .venv      # macOS
python3 -m venv .venv      # Linux
py -m venv .venv           # Windows (PowerShell)
```

Then activate it — **every time you open a new terminal**:

```
source .venv/bin/activate  # macOS
source .venv/bin/activate  # Linux
.venv\Scripts\activate     # Windows (PowerShell)
```

You will see (.venv) at the start of the prompt. If you do not, nothing below will find pandas. Install the packages once, with the environment active:

```
pip install -r requirements.txt
```

Finally tell the editor which Python to use: Ctrl/Cmd + Shift + P, type **Python: Select Interpreter**, and choose the one inside .venv.

A new file, and running it

File > New File, then save it with a .py ending — session03.py, say. Write your code, save it, and in the terminal:

```
python session03.py        # macOS, Linux and Windows alike, inside .venv
```

Inside the environment the command is **python** on all three platforms. Outside it, macOS and Linux need **python3** and Windows needs **py**, which is one more reason to activate first.

For trying one line at a time, type **python** on its own to get the >>> prompt, and **exit()** to leave. The editor's **Run** triangle does the same thing as the command above.

The four things you always do

```
import pandas as pd
```

```
# 1 - build a DataFrame
df = pd.DataFrame({"country": ["FR", "DE", "IT"],
                  "gdp_pc_eur": [37_000, 46_000, 32_000]})
```

```
# 2 - look at what is in it
df.head()      # the first five rows
df.shape       # (rows, columns)
df.dtypes      # what type each column holds
```

```

df.info()          # types, and how many values are missing
df.describe()      # count, mean, sd and quartiles, per numeric column

# 3 - read a file in
df = pd.read_csv("data.csv")
df = pd.read_excel("data.xlsx")          # needs openpyxl
df = pd.read_parquet("data/spine/core.parquet")

import qmib
df = qmib.load("core")                   # a course dataset, by name

# 4 - write a CSV out
df.to_csv("out.csv", index=False)      # index=False, or you get a stray column

```

From session 3

Fit

```

import statsmodels.formula.api as smf
fit = smf.ols("gdp_pc_eur ~ productivity_idx", data=d).fit()
# read the ~ as "explained by"; the intercept is added for you

fit = model.fit()          # .fit() does the arithmetic

```

Read the fit

```

fit.params              # the coefficients, in units of y

fit.rsquared            # share of variance explained

fit.resid, fit.fittedvalues  # what is left, and what was predicted

fit.rsquared_adj        # penalised for each extra predictor

np.sqrt(fit.mse_resid)  # the RSE, in the units of y

```

Diagnose

```

influence = fit.get_influence()  # the whole diagnostic bundle

leverage = influence.hat_matrix_diag  # sums to p, always

cooks_d = influence.cooks_distance[0]  # screen at 4/n

influence.resid_studentized_internal  # on a common scale

```

Choose and validate

```

fit.aic, fit.bic          # lower is better, and only relatively

```

Plot

```

import matplotlib.pyplot as plt
plt.scatter(d["productivity_idx"], d["gdp_pc_eur"], s=12, alpha=0.6)

plt.xlabel("Productivity index"); plt.ylabel("GDP per capita (EUR)")
# say the units, every time

```

```
plt.axhline(0, color="black", lw=1)
plt.axvline(2 * 2 / len(d), color="red", ls="--") # the 2p/n screen

plt.stem(cooks_d) # one spike per observation
```